

-  Procédure d'Installation Production AirBar - Windows
 -  Prérequis Matériel et Logiciel
-  ÉTAPE 1 : Installation de Docker Desktop
 - 1.1 Téléchargement
 - 1.2 Installation
 - 1.3 Configuration post-installation
-  ÉTAPE 2 : Préparation des Fichiers du Serveur
 - 2.1 Installation de Git (si nécessaire)
 - 2.2 Clone du repository backend
 - 2.3 Vérification de la structure
-  ÉTAPE 3 : Configuration pour Production
 - 3.1 Configuration des mots de passe
 - 3.2 Configuration du serveur
 - 3.3 Configuration Docker Compose pour Production
-  ÉTAPE 4 : Déploiement
 - 4.1 Build de l'image Docker
 - 4.2 Lancement des services
 - 4.3 Vérification des logs
-  ÉTAPE 5 : Configuration du Pare-feu Windows
 - 5.1 Ouverture des ports
 - 5.2 Créer les règles pour chaque port
-  ÉTAPE 6 : Validation de l'Installation
 - 6.1 Test de connexion API
 - 6.2 Test depuis un navigateur
 - 6.3 Test de la base de données
 - 6.4 Test depuis l'application Flutter
-  ÉTAPE 7 : Gestion et Maintenance
 - 7.1 Arrêter les services
 - 7.2 Redémarrer les services
 - 7.3 Mettre à jour le serveur
 - 7.4 Voir les logs en temps réel
-  ÉTAPE 8 : Sauvegarde et Restauration
 - 8.1 Sauvegarde de la base de données
 - 8.2 Sauvegarde automatique (planificateur Windows)
 - 8.3 Restauration
-  Dépannage Courant
 - Problème : "Cannot connect to Docker daemon"

- Problème : Port déjà utilisé (8080, 8081, 8082)
- Problème : Serveur ne démarre pas
- Problème : Connexion refusée depuis Flutter
- 🇫🇷 Monitoring et Performance
 - Surveillance des ressources
 - Nettoyage (maintenance)
- 🔒 Sécurité en Production
 - Checklist de sécurité
- 📞 Ressources et Support

User: faire une procédure pas à pas pour installer le serveur en mode production sur une plateforme windows vierge

GitHub Copilot: Je vais vous créer une procédure pas à pas pour installer le serveur AirBar en mode production sur Windows. Laissez-moi d'abord consulter la documentation existante pour m'assurer de couvrir tous les aspects.

Lire []

(file:///Users/philipperobert/development/codage%20en%20flutter/airbar_backend/airbar_backend_server/docker-compose.yaml)

Lire []

(file:///Users/philipperobert/development/codage%20en%20flutter/airbar_backend/airbar_backend_server/Dockerfile)

Lire []

(file:///Users/philipperobert/development/codage%20en%20flutter/airbar/information/guide-docker-dockerfile-compose.md#1-1), lignes 1 à 100

Lire []

(file:///Users/philipperobert/development/codage%20en%20flutter/airbar_backend/airbar_backend_server/config)

Lire []

(file:///Users/philipperobert/development/codage%20en%20flutter/airbar_backend/airbar_backend_server/config/production.yaml)

Voici une procédure complète pas à pas pour installer le serveur AirBar en mode production sur Windows :



Procédure d'Installation Production AirBar - Windows



Prérequis Matériel et Logiciel

Configuration minimale recommandée :

- Windows 10/11 Pro, Enterprise ou Server 2019/2022
 - Processeur : 4 cœurs
 - RAM : 8 GB minimum (16 GB recommandé)
 - Disque : 50 GB d'espace libre (SSD recommandé)
 - Connexion Internet
-



ÉTAPE 1 : Installation de Docker Desktop

1.1 Téléchargement

1. Ouvrir un navigateur et aller sur : <https://www.docker.com/products/docker-desktop/>
2. Cliquer sur **"Download for Windows"**
3. Télécharger **Docker Desktop Installer.exe**

1.2 Installation

1. Exécuter **Docker Desktop Installer.exe** en tant qu'administrateur
2. Dans l'assistant d'installation :
 - Cocher **"Use WSL 2 instead of Hyper-V"** (recommandé)
 - Cocher **"Add shortcut to desktop"**
3. Cliquer sur **"Ok"** et attendre la fin de l'installation
4. Cliquer sur **"Close and restart"**

1.3 Configuration post-installation

1. Après le redémarrage, lancer **Docker Desktop**
2. Accepter les conditions d'utilisation
3. Passer le tutoriel (Skip tutorial)
4. Vérifier que Docker fonctionne :
 - Ouvrir **PowerShell** ou **Invite de commandes**
 - Taper : `docker --version`
 - Résultat attendu : `Docker version 24.x.x, build...`

ÉTAPE 2 : Préparation des Fichiers du Serveur

2.1 Installation de Git (si nécessaire)

1. Télécharger Git : <https://git-scm.com/download/win>
2. Installer avec les options par défaut
3. Redémarrer le terminal

2.2 Clone du repository backend

```
# Créer un dossier pour le projet
cd C:\
mkdir airbar_production
cd airbar_production

# Cloner le repository
git clone https://github.com/rorophil/airbar_backend.git
cd airbar_backend\airbar_backend_server
```

2.3 Vérification de la structure

```
dir
```

Vous devez voir :

- `bin/`
- `config/`
- `lib`
- `migrations/`
- `web`
- `docker-compose.yaml`
- `Dockerfile`
- `pubspec.yaml`

ÉTAPE 3 : Configuration pour Production

3.1 Configuration des mots de passe

1. Ouvrir le fichier `config/passwords.yaml` avec un éditeur de texte (Notepad++)
2. **IMPORTANT : Générer des mots de passe sécurisés**

```
# config/passwords.yaml
production:
  database: 'VOTRE_MOT_DE_PASSE_POSTGRES_SECURISE'
  redis: 'VOTRE_MOT_DE_PASSE_REDIS_SECURISE'
```



Conseil : Utilisez des mots de passe de 32+ caractères aléatoires

3.2 Configuration du serveur

1. Ouvrir `config/production.yaml`
2. Adapter les paramètres selon votre infrastructure :

```
# config/production.yaml
apiServer:
  port: 8080
  publicHost: localhost # ou votre IP publique/nom de domaine
  publicPort: 8080
```

```
publicScheme: http      # https si vous avez un certificat SSL

insightsServer:
  port: 8081
  publicHost: localhost
  publicPort: 8081
  publicScheme: http

webServer:
  port: 8082
  publicHost: localhost
  publicPort: 8082
  publicScheme: http

database:
  host: postgres # Nom du service Docker
  port: 5432
  name: airbar_backend
  user: postgres
  requireSsl: false # true en production avec certificat

redis:
  enabled: true
  host: redis      # Nom du service Docker
  port: 6379

maxRequestSize: 524288

sessionLogs:
  consoleEnabled: true # false en production pour performance
```

3.3 Configuration Docker Compose pour Production

1. Créer un fichier `docker-compose.production.yaml` :

```
version: '3.8'

services:
  # Serveur PostgreSQL
  postgres:
    image: pgvector/pgvector:pg16
    container_name: airbar_postgres
    restart: always
    ports:
      - "5432:5432" # Exposition externe
    environment:
      POSTGRES_USER: postgres
      POSTGRES_DB: airbar_backend
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD} # Depuis .env
    volumes:
```

```
    - postgres_data:/var/lib/postgresql/data
networks:
    - airbar_network
```

```
# Cache Redis
```

```
redis:
  image: redis:6.2.6
  container_name: airbar_redis
  restart: always
  ports:
    - "6379:6379"
  command: redis-server --requirepass ${REDIS_PASSWORD}
  volumes:
    - redis_data:/data
  networks:
    - airbar_network
```

```
# Serveur AirBar
```

```
airbar_server:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: airbar_server
  restart: always
  ports:
    - "8080:8080" # API
    - "8081:8081" # Insights
    - "8082:8082" # Web
  environment:
    runmode: production
    serverid: server-01
    logging: normal
    role: monolith
  depends_on:
    - postgres
    - redis
  networks:
    - airbar_network
  volumes:
    - ./config:/app/config:ro # Configuration en lecture seule
```

```
networks:
  airbar_network:
    driver: bridge
```

```
volumes:
  postgres_data:
    driver: local
  redis_data:
    driver: local
```

2. Créer un fichier `.env` dans le même dossier :

```
# .env
POSTGRES_PASSWORD=VOTRE_MOT_DE_PASSE_POSTGRES_SECURISE
REDIS_PASSWORD=VOTRE_MOT_DE_PASSE_REDIS_SECURISE
```

ÉTAPE 4 : Déploiement

4.1 Build de l'image Docker

```
# Naviguer vers le dossier du serveur
cd C:\airbar_production\airbar_backend\airbar_backend_server

# Build de l'image (peut prendre 5-10 minutes)
docker build -t airbar_server:latest .
```

Vérifier la création de l'image :

```
docker images
```

Vous devez voir `airbar_server` avec le tag `latest`

4.2 Lancement des services

```
# Démarrer tous les services en arrière-plan
docker-compose -f docker-compose.production.yaml up -d

# Vérifier que les conteneurs tournent
docker ps
```

Vous devez voir 3 conteneurs :

- `airbar_postgres` (Up)
- `airbar_redis` (Up)
- `airbar_server` (Up)

4.3 Vérification des logs

```
# Logs du serveur AirBar
docker logs -f airbar_server

# Logs PostgreSQL
docker logs airbar_postgres

# Logs Redis
docker logs airbar_redis
```

Messages de succès attendus :

```
Starting Serverpod...
Server 'airbar_backend' listening on port 8080
Migrations applied successfully
Server ready to accept connections
```

ÉTAPE 5 : Configuration du Pare-feu Windows

5.1 Ouverture des ports

1. Ouvrir **Panneau de configuration** → **Système et sécurité** → **Pare-feu Windows Defender**
2. Cliquer sur "**Paramètres avancés**" (dans le menu de gauche)
3. Cliquer sur "**Règles de trafic entrant**"

5.2 Créer les règles pour chaque port

Pour le port 8080 (API) :

1. Clic droit → "**Nouvelle règle...**"
2. Type de règle : "**Port**" → Suivant
3. **TCP** → Ports locaux spécifiques : **8080** → Suivant

4. **Autoriser la connexion** → Suivant
5. Profils : ☒ Domaine ☒ Privé ☒ Public → Suivant
6. Nom : **"AirBar API Server"** → Terminer

Répéter pour :

- Port **8081** : "AirBar Insights Server"
- Port **8082** : "AirBar Web Server"
- Port **5432** : "PostgreSQL AirBar" (si accès externe requis)
- Port **6379** : "Redis AirBar" (si accès externe requis)

 **Sécurité** : En production, ne pas exposer directement PostgreSQL et Redis à l'extérieur

ÉTAPE 6 : Validation de l'Installation

6.1 Test de connexion API

```
# Tester l'endpoint de santé
curl http://localhost:8080/

# Devrait retourner une réponse du serveur Serverpod
```

6.2 Test depuis un navigateur

1. Ouvrir un navigateur
2. Aller sur : <http://localhost:8082>
3. Vous devriez voir l'interface web Serverpod

6.3 Test de la base de données

```
# Se connecter à PostgreSQL
docker exec -it airbar_postgres psql -U postgres -d airbar_backend

# Dans psql, lister les tables :
\dt
```

```
# Vérifier la table users
SELECT * FROM users;

# Quitter
\q
```

6.4 Test depuis l'application Flutter

1. Sur le même réseau, noter l'adresse IP du serveur Windows :

```
ipconfig
```

Chercher **"Adresse IPv4"** (ex: 192.168.1.100)

2. Dans l'application Flutter (écran de login) :

- Cliquer sur **"Configuration serveur"**
- Entrer : **192.168.1.100:8080** (remplacer par votre IP)
- Tester la connexion
- Sauvegarder

3. Tenter une connexion avec :

- PIN par défaut : **123456**
- Rôle : admin

ÉTAPE 7 : Gestion et Maintenance

7.1 Arrêter les services

```
docker-compose -f docker-compose.production.yaml down
```

7.2 Redémarrer les services

```
docker-compose -f docker-compose.production.yaml restart
```

7.3 Mettre à jour le serveur

```
# 1. Arrêter les services
docker-compose -f docker-compose.production.yaml down

# 2. Récupérer les dernières modifications
git pull origin main

# 3. Rebuild l'image
docker build -t airbar_server:latest .

# 4. Redémarrer
docker-compose -f docker-compose.production.yaml up -d
```

7.4 Voir les logs en temps réel

```
docker-compose -f docker-compose.production.yaml logs -f
```

ÉTAPE 8 : Sauvegarde et Restauration

8.1 Sauvegarde de la base de données

```
# Créer un dossier de sauvegarde
mkdir C:\airbar_production\backups

# Exporter la base de données
docker exec airbar_postgres pg_dump -U postgres airbar_backend >
C:\airbar_production\backups\backup_%date%.sql
```

8.2 Sauvegarde automatique (planificateur Windows)

1. Créer un script `backup.bat` :

```
@echo off
set BACKUP_DIR=C:\airbar_production\backups
set
TIMESTAMP=%date:~-4%%date:~3,2%%date:~0,2%_%time:~0,2%%time:~3,2%%time:~6,2%
docker exec airbar_postgres pg_dump -U postgres airbar_backend >
%BACKUP_DIR%\backup_%TIMESTAMP%.sql
```

2. Ouvrir **Planificateur de tâches**
3. Créer une tâche quotidienne exécutant `backup.bat`

8.3 Restauration

```
# Arrêter le serveur
docker-compose -f docker-compose.production.yaml stop airbar_server

# Restaurer la sauvegarde
type C:\airbar_production\backups\backup_YYYYMMDD.sql | docker exec -i
airbar_postgres psql -U postgres -d airbar_backend

# Redémarrer
docker-compose -f docker-compose.production.yaml start airbar_server
```



Dépannage Courant

Problème : "Cannot connect to Docker daemon"

Solution : Lancer Docker Desktop et attendre qu'il soit complètement démarré

Problème : Port déjà utilisé (8080, 8081, 8082)

```
# Identifier le processus utilisant le port
netstat -ano | findstr :8080

# Terminer le processus (remplacer PID par le numéro trouvé)
taskkill /PID <PID> /F
```

Problème : Serveur ne démarre pas

```
# Vérifier les logs
docker logs airbar_server

# Vérifier les variables d'environnement
docker exec airbar_server env

# Redémarrer en mode verbose
docker-compose -f docker-compose.production.yaml up
```

Problème : Connexion refusée depuis Flutter

1. Vérifier le pare-feu Windows
2. Vérifier que les conteneurs sont actifs : `docker ps`
3. Tester depuis le serveur : `curl http://localhost:8080`
4. Vérifier l'IP du serveur : `ipconfig`



Monitoring et Performance

Surveillance des ressources

```
# Stats des conteneurs
docker stats

# Espace disque utilisé
docker system df

# Logs de performance
docker logs airbar_server | findstr "performance|slow|error"
```

Nettoyage (maintenance)

```
# Supprimer les images non utilisées
docker image prune -a

# Supprimer les volumes non utilisés
docker volume prune

# Nettoyage complet (ATTENTION : sauvegardez avant)
docker system prune -a --volumes
```



Sécurité en Production

Checklist de sécurité

- ☐ Mots de passe forts dans `.env` et `passwords.yaml`
- ☐ Fichier `.env` dans `.gitignore`
- ☐ Pare-feu configuré (ports limités)
- ☐ SSL/HTTPS configuré (certificat Let's Encrypt recommandé)
- ☐ PostgreSQL et Redis non exposés publiquement
- ☐ Sauvegardes automatiques configurées
- ☐ Windows Update actif
- ☐ Antivirus/Windows Defender actif
- ☐ Accès RDP sécurisé (si accès distant)



Ressources et Support

- **Documentation Serverpod** : <https://serverpod.dev/S>
- **Docker Documentation** : <https://docs.docker.com/>
- **PostgreSQL** : <https://www.postgresql.org/docs/>
- **Redis** : <https://redis.io/documentation>

✅ **Installation terminée !** Votre serveur AirBar est maintenant en mode production sur Windows.